Practical 1

1. Perform basic file and directory operations using Linux command-line utilities. Tasks: i. Display the current working directory and list files, including hidden files. ii. Create a directory named OS_Lab and three subdirectories inside it. iii. Create text files, write content into them, and display their contents. iv. Copy, rename, move, and delete files using appropriate commands. v. Search for a specified file and display its size and other file information.

```
# i. Display current working directory and list all files including hidden files

pwd
ls -la

# ii. Create OS_Lab directory and three subdirectories

mkdir -p OS_Lab
cd OS_Lab
mkdir -p FY SY TE

# iii. Create text files and write content into them

echo "Operating System Lab - FY Students" > FY/fy.txt
echo "Operating System Lab - SY Students" > SY/sy.txt
echo "Operating System Lab - TE Students" > TE/te.txt

# Display file contents

cat FY/fy.txt
cat SY/sy.txt
cat TE/te.txt

# iv. Copy, rename, move and delete files

# Copy file
cp FY/fy.txt SY/

# Rename file
mv SY/fy.txt SY/fy_records.txt

# Move file
mv TE/te.txt OS_Lab_te.txt

# Delete file
rm SY/fy_records.txt

# v. Search for a specified file
```

```
find . -name "sy.txt"

# Display file size and other information

stat SY/sy.txt



# Display final directory structure

ls -R
```

practical 2

2. Manage file permissions and ownership, monitor running processes, and inspect system resource usage in Linux. Tasks: i. Create files with different permissions and display their permission details. ii. Modify read, write, and execute permissions using symbolic and numeric modes. iii. Identify the current user and group, and demonstrate ownership changes where administrator privileges are available. iv. Display running processes and identify their process IDs. v. Monitor CPU and memory usage and display disk-space information.

```
# i. Create files with different permissions and display permission details

mkdir -p OS_Permissions
cd OS_Permissions

touch file1.txt file2.txt file3.txt

# Set different permissions
chmod 644 file1.txt
chmod 600 file2.txt
chmod 755 file3.txt

# Display permission details
ls -l

# ii. Modify read, write and execute permissions

# Symbolic mode
chmod u+x file1.txt
chmod g+w file1.txt
chmod o-r file1.txt
```

```
# Numeric mode
chmod 755 file2.txt
chmod 644 file3.txt

# Display updated permissions
ls -l

# iii. Identify current user and group

whoami
id
groups

# Demonstrate ownership information
ls -l file1.txt

# Change ownership (administrator privileges required)
# Replace mukund with your Ubuntu username if different
sudo chown mukund file1.txt

# Change group ownership
sudo chgrp mukund file1.txt

# Display updated ownership
ls -l file1.txt

# iv. Display running processes and identify their PIDs

ps
ps -ef
ps -u "$USER"

# Display process IDs
pgrep -a bash

# v. Monitor CPU and memory usage

# Display CPU and memory information
top -b -n 1 | head -15

# Display memory usage
free -h
```

```
# Display disk-space information
df -h

# Display directory size
du -sh .

# Display final file permissions
ls -l
```

Practical 3

3. Create a directory named Student_Records. Perform the following operations: i. Create separate directories for FY, SY, TE and BE students. ii. Create text files containing student names and roll numbers. iii. Copy selected files into another directory. iv. Rename and delete specified files. v. Display the complete directory structure.

```
# Step 1: Create main directory
mkdir -p Student_Records
cd Student_Records

# Step 2: Create separate directories
mkdir -p FY SY TE BE

# Step 3: Create student text files
echo -e "Name: Rahul\nRoll No: 101\nName: Amit\nRoll No: 102" > FY/FY_Students.txt

echo -e "Name: Soham\nRoll No: 201\nName: Mukund\nRoll No: 202" > SY/SY_Students.txt

echo -e "Name: Guru\nRoll No: 301\nName: Akash\nRoll No: 302" > TE/TE_Students.txt

echo -e "Name: Priya\nRoll No: 401\nName: Sneha\nRoll No: 402" > BE/BE_Students.txt

# Step 4: Create directory and copy selected files
mkdir -p Selected_Students

cp FY/FY_Students.txt Selected_Students/
cp TE/TE_Students.txt Selected_Students/
```

```
# Step 5: Rename FY file
mv FY/FY_Students.txt FY/FY_Records.txt

# Step 6: Delete BE student file
rm BE/BE_Students.txt

# Step 7: Install tree command
sudo apt install tree -y

# Step 8: Display complete directory structure
cd ..
tree Student_Records
```

PRACTICAL -4

4. Create a directory named Student_Records. Perform the following operations: i. Create separate directories for FY, SY, TE and BE students. ii. Create text files containing student names and roll numbers. iii. Copy selected files into another directory. iv. Rename and delete specified files. v. Display the complete directory structure.

```
# Create project directory
mkdir -p AIML_Project
cd AIML_Project

# i. Create files for students, faculty and administrators
touch students.txt faculty.txt administrators.txt

# Write content into files
echo "Student Name: Mukund, Roll No: 101" > students.txt
echo "Faculty Name: Professor Renuka" > faculty.txt
echo "Administrator: System Admin" > administrators.txt

# ii. Assign different permissions to files
chmod 644 students.txt
chmod 640 faculty.txt
chmod 600 administrators.txt

# iii. Display file ownership and permission details
```

```
ls -l

# Display current user and group
whoami
id

# iv. Change permissions using symbolic notation
chmod u+x students.txt
chmod g+w faculty.txt
chmod o-r faculty.txt

# Display updated permissions
ls -l

# Change permissions using numeric notation
chmod 755 students.txt
chmod 644 faculty.txt
chmod 600 administrators.txt

# Display final permissions
ls -l


# v. Demonstrate read permission
echo "Reading student file:"
cat students.txt

# Demonstrate write permission
echo "New Student: Rahul, Roll No: 102" >> students.txt
echo "Updated student file:"
cat students.txt

# Demonstrate execute permission
echo '#!/bin/bash' > script.sh
echo 'echo "AIML Project Script Executed Successfully!"' >> script.sh

# Assign execute permission
chmod 755 script.sh

# Execute the script
./script.sh
```

```
# Demonstrate removal of read permission
chmod 000 students.txt

# Try reading the file (expected: Permission denied)
cat students.txt

# Restore normal permissions
chmod 644 students.txt

# Display final ownership and permission details
ls -l
```

PRACTICAL-5

5. Write a shell script to- i. to generate a multiplication table using loops (for, while, until) ii. to check whether a number is positive, negative or zero, using if-elif-else. iii. Develop a script to check whether a file exists, display its size and basic information.

1)CREATE FILE
nano practical.sh

```
#!/bin/bash

# i. Multiplication table using FOR loop

echo "Enter a number for FOR loop:"
read num

echo "Multiplication table using FOR loop:"
for ((i=1; i<=10; i++))
```

```
do
   echo "$num x $i = $((num*i))"
done

# Multiplication table using WHILE loop

echo " "
echo "Multiplication table using WHILE loop:"
i=1

while [ $i -le 10 ]
do
   echo "$num x $i = $((num*i))"
   i=$((i+1))
done

# Multiplication table using UNTIL loop

echo ""
echo "Multiplication table using UNTIL loop:"
i=1

until [ $i -gt 10 ]
do
   echo "$num x $i = $((num*i))"
   i=$((i+1))
done

# ii. Check positive, negative or zero using if-elif-else

echo ""
echo "Enter a number to check:"
read n

if [ $n -gt 0 ]
then
   echo "Number is Positive"
elif [ $n -lt 0 ]
then
   echo "Number is Negative"
else
   echo "Number is Zero"
```

```
fi

# iii. Check whether a file exists

echo ""
echo "Enter file name:"
read filename

if [ -e "$filename" ]
then
   echo "File exists"
   echo "File information:"
   ls -l "$filename"
   echo "File size:"
   du -h "$filename"
else
   echo "File does not exist"
fi
```

Step 2: Give execute permission

```
chmod +x practical.sh
```

Step 3: Run the script

```
./practical.sh
```

PRACTICAL -6

6. Write a shell script to- i. to generate calculate the factorial of a number using loops (for, while, until). ii. to check whether a number is even or odd. using if –else. iii. display a number entered by user using function

```
#!/bin/bash

# i. Calculate factorial using FOR loop

echo "Enter a number for factorial:"
read num

fact=1

for ((i=1; i<=num; i++))
```

```
do
   fact=$((fact*i))
done

echo "Factorial using FOR loop = $fact"


# Calculate factorial using WHILE loop

fact=1
i=1

while [ $i -le $num ]
do
   fact=$((fact*i))
   i=$((i+1))
done

echo "Factorial using WHILE loop = $fact"


# Calculate factorial using UNTIL loop

fact=1
i=1

until [ $i -gt $num ]
do
   fact=$((fact*i))
   i=$((i+1))
done

echo "Factorial using UNTIL loop = $fact"


# ii. Check whether number is even or odd

echo ""
echo "Enter a number to check even or odd:"
read n

if [ $((n%2)) -eq 0 ]
then
   echo "$n is Even"
else
   echo "$n is Odd"
fi
```

```
# iii. Display number using function

display_number()
{
   echo "The entered number is: $1"
}

echo ""
echo "Enter a number to display:"
read value

display_number "$value"
```

PRACTICAL – 7

7. Develop programs to create and manage processes using system calls. Implement CPU scheduling algorithms (FCFS, SJF) and analyze their impact on process execution, memory utilization, and overall system performance.

1)Create a file named:

```
nano practical.c
```

```
#include <stdio.h>
#include <stdlib.h>
```

```c
#include <unistd.h>
#include <sys/wait.h>

struct Process {
    int pid, bt, wt, tat;
};

int main() {
    int n, i, j;
    struct Process p[20], temp;
    float avgWT = 0, avgTAT = 0;

    // Process creation using fork()
    printf("PROCESS MANAGEMENT\n");

    pid_t pid = fork();

    if (pid < 0) {
        printf("Process creation failed\n");
        return 1;
    }
    else if (pid == 0) {
        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());
        exit(0);
    }
    else {
        wait(NULL);
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
    }

    // Input processes
    printf("\nCPU SCHEDULING\n");
    printf("Enter number of processes: ");
    scanf("%d", &n);

    if (n < 1 || n > 20) {
        printf("Enter processes between 1 and 20\n");
        return 1;
    }

    for (i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("Enter Burst Time for P%d: ", i + 1);
        scanf("%d", &p[i].bt);
        if (p[i].bt <= 0) {
            printf("Burst time must be positive\n");
            return 1;
        }
    }

    // FCFS Scheduling
    printf("\nFCFS SCHEDULING\n");
```

```
p[0].wt = 0;

for (i = 1; i < n; i++) {
    p[i].wt = p[i-1].wt + p[i-1].bt;
}

avgWT = 0;
avgTAT = 0;

printf("\nPID\tBT\tWT\tTAT\n");

for (i = 0; i < n; i++) {
    p[i].tat = p[i].wt + p[i].bt;
    avgWT += p[i].wt;
    avgTAT += p[i].tat;

    printf("P%d\t%d\t%d\t%d\n",
           p[i].pid, p[i].bt, p[i].wt, p[i].tat);
}

printf("Average Waiting Time = %.2f\n", avgWT/n);
printf("Average Turnaround Time = %.2f\n", avgTAT/n);

// SJF Scheduling (Non-preemptive)
printf("\nSJF SCHEDULING\n");

// Sort processes by burst time
for (i = 0; i < n-1; i++) {
    for (j = i+1; j < n; j++) {
        if (p[i].bt > p[j].bt) {
            temp = p[i];
            p[i] = p[j];
            p[j] = temp;
        }
    }
}

p[0].wt = 0;

for (i = 1; i < n; i++) {
    p[i].wt = p[i-1].wt + p[i-1].bt;
}

avgWT = 0;
avgTAT = 0;

printf("\nPID\tBT\tWT\tTAT\n");

for (i = 0; i < n; i++) {
    p[i].tat = p[i].wt + p[i].bt;
    avgWT += p[i].wt;
    avgTAT += p[i].tat;

    printf("P%d\t%d\t%d\t%d\n",
           p[i].pid, p[i].bt, p[i].wt, p[i].tat);
}
```

```
    printf("Average Waiting Time = %.2f\n", avgWT/n);
    printf("Average Turnaround Time = %.2f\n", avgTAT/n);

    return 0;
}
```

Step 1: Compile

```
gcc practical.c -o practical
```

Step 2: Run

```
./practical
```

PRACTICAL -8

7. Develop programs to create and manage processes using system calls. Implement CPU scheduling algorithms (SJF, Round Robin) and analyze their impact on process execution, memory utilization, and overall system performance.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
```

```
#include <stdlib.h>

int main()
{
   int n, i, j, tq;
   int bt[20], wt[20], tat[20];
   int rem[20], p[20], temp;
   float avgWT, avgTAT;

   // Process creation using fork()
   printf("PROCESS MANAGEMENT\n");

   pid_t pid = fork();

   if (pid == 0)
   {
      printf("\nChild Process\n");
      printf("Child PID: %d\n", getpid());
      printf("Parent PID: %d\n", getppid());
      exit(0);
   }
   else if (pid > 0)
   {
      wait(NULL);
      printf("\nParent Process\n");
      printf("Parent PID: %d\n", getpid());
      printf("Child PID: %d\n", pid);
   }
   else
   {
      printf("Fork failed\n");
      return 1;
   }

   // Input processes
   printf("\nEnter number of processes: ");
   scanf("%d", &n);

   if (n < 1 || n > 20)
   {
      printf("Invalid number of processes\n");
      return 1;
   }

   for (i = 0; i < n; i++)
   {
      p[i] = i + 1;
      printf("Enter burst time for P%d: ", i + 1);
```

```
    scanf("%d", &bt[i]);

    if (bt[i] <= 0)
    {
        printf("Burst time must be positive\n");
        return 1;
    }
}

// SJF Scheduling
printf("\nSJF SCHEDULING\n");

int sjfBT[20], sjfP[20];

for (i = 0; i < n; i++)
{
    sjfBT[i] = bt[i];
    sjfP[i] = p[i];
}

// Sort according to burst time
for (i = 0; i < n - 1; i++)
{
    for (j = i + 1; j < n; j++)
    {
        if (sjfBT[i] > sjfBT[j])
        {
            temp = sjfBT[i];
            sjfBT[i] = sjfBT[j];
            sjfBT[j] = temp;

            temp = sjfP[i];
            sjfP[i] = sjfP[j];
            sjfP[j] = temp;
        }
    }
}

wt[0] = 0;

for (i = 1; i < n; i++)
    wt[i] = wt[i - 1] + sjfBT[i - 1];

avgWT = 0;
avgTAT = 0;

printf("\nPID\tBT\tWT\tTAT\n");
```

```
for (i = 0; i < n; i++)
{
   tat[i] = wt[i] + sjfBT[i];
   avgWT += wt[i];
   avgTAT += tat[i];

   printf("P%d\t%d\t%d\t%d\n",
         sjfP[i], sjfBT[i], wt[i], tat[i]);
}

printf("Average Waiting Time = %.2f\n", avgWT / n);
printf("Average Turnaround Time = %.2f\n", avgTAT / n);

// Round Robin Scheduling
printf("\nROUND ROBIN SCHEDULING\n");

printf("Enter Time Quantum: ");
scanf("%d", &tq);

if (tq <= 0)
{
   printf("Time quantum must be positive\n");
   return 1;
}

for (i = 0; i < n; i++)
   rem[i] = bt[i];

int time = 0;
int completed = 0;

while (completed < n)
{
   for (i = 0; i < n; i++)
   {
      if (rem[i] > 0)
      {
         if (rem[i] > tq)
         {
            time += tq;
            rem[i] -= tq;
         }
         else
         {
            time += rem[i];
            wt[i] = time - bt[i];
            rem[i] = 0;
            completed++;
```

```
            }
          }
        }
    }

    avgWT = 0;
    avgTAT = 0;

    printf("\nPID\tBT\tWT\tTAT\n");

    for (i = 0; i < n; i++)
    {
        tat[i] = wt[i] + bt[i];
        avgWT += wt[i];
        avgTAT += tat[i];

        printf("P%d\t%d\t%d\t%d\n",
              p[i], bt[i], wt[i], tat[i]);
    }

    printf("Average Waiting Time = %.2f\n", avgWT / n);
    printf("Average Turnaround Time = %.2f\n", avgTAT / n);

    return 0;
}
```

Compile:

```
gcc practical8.c -o practical8
```

Run:

```
./practical8
```

PRACTICAL-9

9. Develop programs to create and manage processes using system calls. Implement CPU scheduling algorithms (FCFS, Round Robin) and analyze their impact on process execution, memory utilization, and overall system performance.

Create a file named `practical9.c` in VS Code.

```
nano practical9.c
```

Paste this code:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    int n, i, tq, time = 0;
    int bt[20], wt[20], tat[20], rem[20];
    float avgWT, avgTAT;

    // Process Management using fork()
    printf("PROCESS MANAGEMENT\n");

    pid_t pid = fork();

    if (pid < 0)
    {
        printf("Process creation failed\n");
        return 1;
    }
    else if (pid == 0)
    {
        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());
        exit(0);
    }
    else
    {
        wait(NULL);
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
    }

    // Input number of processes
    printf("\nEnter number of processes: ");
    scanf("%d", &n);

    if (n < 1 || n > 20)
    {
```

```
    printf("Invalid number of processes\n");
    return 1;
}

for (i = 0; i < n; i++)
{
    printf("Enter burst time for P%d: ", i + 1);
    scanf("%d", &bt[i]);

    if (bt[i] <= 0)
    {
        printf("Burst time must be positive\n");
        return 1;
    }

    rem[i] = bt[i];
}

// FCFS Scheduling
printf("\nFCFS SCHEDULING\n");

wt[0] = 0;

for (i = 1; i < n; i++)
{
    wt[i] = wt[i - 1] + bt[i - 1];
}

avgWT = 0;
avgTAT = 0;

printf("\nPID\tBT\tWT\tTAT\n");

for (i = 0; i < n; i++)
{
    tat[i] = wt[i] + bt[i];
    avgWT += wt[i];
    avgTAT += tat[i];

    printf("P%d\t%d\t%d\t%d\n",
           i + 1, bt[i], wt[i], tat[i]);
}

printf("Average Waiting Time = %.2f\n", avgWT / n);
printf("Average Turnaround Time = %.2f\n", avgTAT / n);

// Round Robin Scheduling
printf("\nROUND ROBIN SCHEDULING\n");

printf("Enter Time Quantum: ");
scanf("%d", &tq);

if (tq <= 0)
{
    printf("Time quantum must be positive\n");
    return 1;
}
```

```
    time = 0;
    int completed = 0;

    while (completed < n)
    {
        for (i = 0; i < n; i++)
        {
            if (rem[i] > 0)
            {
                if (rem[i] > tq)
                {
                    time += tq;
                    rem[i] -= tq;
                }
                else
                {
                    time += rem[i];
                    wt[i] = time - bt[i];
                    rem[i] = 0;
                    completed++;
                }
            }
        }
    }

    avgWT = 0;
    avgTAT = 0;

    printf("\nPID\tBT\tWT\tTAT\n");

    for (i = 0; i < n; i++)
    {
        tat[i] = wt[i] + bt[i];
        avgWT += wt[i];
        avgTAT += tat[i];

        printf("P%d\t%d\t%d\t%d\n",
               i + 1, bt[i], wt[i], tat[i]);
    }

    printf("Average Waiting Time = %.2f\n", avgWT / n);
    printf("Average Turnaround Time = %.2f\n", avgTAT / n);

    return 0;
}
```

2. Compile and run

Compile the program: `gcc practical9.c -o practical9`

Run:

```
./practical9
```

Practical 10:
Practical 10
10. Implement Inter-Process Communication using pipes and shared memory. Analyze communication efficiency, resource sharing, and memory utilization.

Create a file named `practical10.c`:

```
nano practical10.c
```

Paste the following code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/ipc.h>
#include <sys/shm.h>

int main()
{
    int fd[2];
    pid_t pid;
    char message[] = "Hello from Parent Process!";
    char buffer[100];

    // IPC USING PIPE
    printf("\n--- IPC USING PIPE ---\n");

    if (pipe(fd) == -1)
    {
        perror("Pipe failed");
        return 1;
    }

    pid = fork();

    if (pid < 0)
    {
        perror("Fork failed");
        return 1;
    }

    if (pid == 0)
    {
        // Child process reads data
        close(fd[1]);

        read(fd[0], buffer, sizeof(buffer));
        printf("Child received through pipe: %s\n", buffer);

        close(fd[0]);
```

```
    exit(0);
}
else
{
    // Parent process writes data
    close(fd[0]);

    write(fd[1], message, strlen(message) + 1);
    close(fd[1]);

    wait(NULL);
    printf("Parent sent message through pipe.\n");
}

// IPC USING SHARED MEMORY
printf("\n--- IPC USING SHARED MEMORY ---\n");

int shmid;
char *shared_memory;

// Create shared memory
shmid = shmget(IPC_PRIVATE, 1024, IPC_CREAT | 0666);

if (shmid < 0)
{
    perror("Shared memory creation failed");
    return 1;
}

// Attach shared memory
shared_memory = (char *)shmat(shmid, NULL, 0);

if (shared_memory == (char *)-1)
{
    perror("Shared memory attachment failed");
    shmctl(shmid, IPC_RMID, NULL);
    return 1;
}

// Parent writes data into shared memory
strcpy(shared_memory, "Hello from Shared Memory!");

pid = fork();

if (pid < 0)
{
    perror("Fork failed");
    shmdt(shared_memory);
    shmctl(shmid, IPC_RMID, NULL);
    return 1;
}

if (pid == 0)
{
    // Child reads shared memory
    printf("Child received through shared memory: %s\n",
           shared_memory);
```

```
        shmdt(shared_memory);
        exit(0);
    }
    else
    {
        wait(NULL);

        printf("Parent shared data successfully.\n");

        // Detach and remove shared memory
        shmdt(shared_memory);
        shmctl(shmid, IPC_RMID, NULL);
    }

    printf("\nIPC Demonstration Completed Successfully!\n");

    return 0;
}
```

2. Compile and run the program

Step 1: Save the file

- Press `Ctrl + S` in VS Code.

Step 2: Compile

```
gcc practical10.c -o practical10
```

Step 3: Execute

```
./practical10
```

Practical 11

11. Implement Inter-Process Communication using pipes and message queues. Analyze communication efficiency, resource sharing, and memory utilization.

Create a file named `practical11.c`:

```
nano practical11.c
```

Paste this complete code:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/ipc.h>
#include <sys/msg.h>

struct message {
    long type;
    char text[100];
};

int main()
{
    int fd[2];
    pid_t pid;
    char msg[] = "Hello from Parent using Pipe!";
    char buffer[100];

    // IPC USING PIPE
    printf("\n--- IPC USING PIPE ---\n");

    if (pipe(fd) == -1) {
        perror("Pipe creation failed");
        return 1;
    }

    pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        return 1;
    }

    if (pid == 0) {
        // Child reads message
        close(fd[1]);

        if (read(fd[0], buffer, sizeof(buffer)) > 0) {
            printf("Child received: %s\n", buffer);
        }

        close(fd[0]);
        exit(0);
    }
    else {
        // Parent writes message
        close(fd[0]);

        write(fd[1], msg, strlen(msg) + 1);
        close(fd[1]);

        wait(NULL);
        printf("Parent sent message using Pipe.\n");
    }
```

```
    // IPC USING MESSAGE QUEUE
    printf("\n--- IPC USING MESSAGE QUEUE ---\n");

    int qid;
    struct message m;

    // Create message queue
    qid = msgget(IPC_PRIVATE, IPC_CREAT | 0666);

    if (qid == -1) {
        perror("Message queue creation failed");
        return 1;
    }

    pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        msgctl(qid, IPC_RMID, NULL);
        return 1;
    }

    if (pid == 0) {
        // Child receives message
        if (msgrcv(qid, &m, sizeof(m.text), 1, 0) == -1) {
            perror("Message receive failed");
            exit(1);
        }

        printf("Child received: %s\n", m.text);
        exit(0);
    }
    else {
        // Parent sends message
        m.type = 1;
        strcpy(m.text, "Hello from Parent using Message Queue!");

        if (msgsnd(qid, &m, sizeof(m.text), 0) == -1) {
            perror("Message send failed");
            wait(NULL);
            msgctl(qid, IPC_RMID, NULL);
            return 1;
        }

        wait(NULL);
        printf("Parent sent message using Message Queue.\n");

        // Remove message queue
        msgctl(qid, IPC_RMID, NULL);
    }

    printf("\nIPC Communication Completed Successfully!\n");

    return 0;
}
```

2. Compile and run the program

Step 1: Save your file

Press `Ctrl + S` in VS Code.

Step 2: Compile

```
gcc practical11.c -o practical11
```

Step 3: Run

```
./practical11
```

12. Implement Inter-Process Communication using shared memory and message queues. Analyze communication efficiency, resource sharing, and memory utilization.

Create a file named `practical12.c`:

```
nano practical12.c
```

Paste this complete code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/msg.h>

// Message Queue Structure
struct message {
    long type;
    char text[100];
};

int main()
{
```

```
pid_t pid;
int shmid;
char *shared_memory;

// IPC USING SHARED MEMORY
printf("\n--- IPC USING SHARED MEMORY ---\n");

// Create shared memory
shmid = shmget(IPC_PRIVATE, 1024, IPC_CREAT | 0666);

if (shmid == -1) {
    perror("Shared memory creation failed");
    return 1;
}

// Attach shared memory
shared_memory = (char *)shmat(shmid, NULL, 0);

if (shared_memory == (char *)-1) {
    perror("Shared memory attachment failed");
    shmctl(shmid, IPC_RMID, NULL);
    return 1;
}

// Parent writes message
strcpy(shared_memory, "Hello from Shared Memory!");

pid = fork();

if (pid < 0) {
    perror("Fork failed");
    shmdt(shared_memory);
    shmctl(shmid, IPC_RMID, NULL);
    return 1;
}

if (pid == 0) {
    // Child reads shared memory
    printf("Child received: %s\n", shared_memory);

    shmdt(shared_memory);
    exit(0);
}
else {
    wait(NULL);

    printf("Parent shared data successfully.\n");

    // Detach and remove shared memory
    shmdt(shared_memory);
    shmctl(shmid, IPC_RMID, NULL);
}

// IPC USING MESSAGE QUEUE
printf("\n--- IPC USING MESSAGE QUEUE ---\n");

int qid;
```

```
    struct message m;

    // Create message queue
    qid = msgget(IPC_PRIVATE, IPC_CREAT | 0666);

    if (qid == -1) {
        perror("Message queue creation failed");
        return 1;
    }

    pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        msgctl(qid, IPC_RMID, NULL);
        return 1;
    }

    if (pid == 0) {
        // Child receives message
        if (msgrcv(qid, &m, sizeof(m.text), 1, 0) == -1) {
            perror("Message receive failed");
            exit(1);
        }

        printf("Child received: %s\n", m.text);
        exit(0);
    }
    else {
        // Parent sends message
        m.type = 1;
        strcpy(m.text, "Hello from Message Queue!");

        if (msgsnd(qid, &m, sizeof(m.text), 0) == -1) {
            perror("Message send failed");
            wait(NULL);
            msgctl(qid, IPC_RMID, NULL);
            return 1;
        }

        wait(NULL);

        printf("Parent sent message successfully.\n");

        // Remove message queue
        msgctl(qid, IPC_RMID, NULL);
    }

    printf("\nIPC Demonstration Completed Successfully!\n");

    return 0;
}
```

2. Compile and run the program

Step 1: Save the file

Press `Ctrl + S` in VS Code.

Step 2: Compile

```
gcc practical12.c -o practical12
```

Step 3: Execute

```
./practical12
```

13. Create a multithreaded application using POSIX threads to execute concurrent tasks. Measure CPU utilization, memory consumption, and execution time under different thread

Save the file as `practical13.c`.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <time.h>
#include <sys/resource.h>
#include <unistd.h>

#define TOTAL_WORK 50000000

void *calculate(void *arg)
{
    long thread_id = *(long *)arg;
    long long sum = 0;

    for (long i = 1; i <= TOTAL_WORK; i++)
    {
        sum += i % 100;
    }

    printf("Thread %ld completed. Result = %lld\n",
           thread_id, sum);

    return NULL;
}

int main()
{
```

```
int thread_counts[] = {1, 2, 4, 8};
int tests = 4;

printf("MULTITHREADING PERFORMANCE ANALYSIS\n");

for (int t = 0; t < tests; t++)
{
    int n = thread_counts[t];

    pthread_t threads[n];
    long ids[n];

    struct timespec start, end;
    struct rusage usage;

    printf("\n----------------------------------\n");
    printf("Number of Threads: %d\n", n);

    clock_gettime(CLOCK_MONOTONIC, &start);

    for (int i = 0; i < n; i++)
    {
        ids[i] = i + 1;

        if (pthread_create(&threads[i], NULL,
                           calculate, &ids[i]) != 0)
        {
            perror("Thread creation failed");
            exit(1);
        }
    }

    for (int i = 0; i < n; i++)
    {
        pthread_join(threads[i], NULL);
    }

    clock_gettime(CLOCK_MONOTONIC, &end);

    double elapsed =
        (end.tv_sec - start.tv_sec) +
        (end.tv_nsec - start.tv_nsec) / 1000000000.0;

    getrusage(RUSAGE_SELF, &usage);

    double cpu_time =
        usage.ru_utime.tv_sec +
        usage.ru_utime.tv_usec / 1000000.0 +
        usage.ru_stime.tv_sec +
        usage.ru_stime.tv_usec / 1000000.0;

    double cpu_utilization =
        (cpu_time / elapsed) * 100.0;

    long memory_kb = usage.ru_maxrss;

    printf("\nPerformance Results:\n");
```

```
        printf("Execution Time: %.3f seconds\n", elapsed);
        printf("CPU Utilization: %.2f%% (approx.)\n",
               cpu_utilization);
        printf("Maximum Resident Memory: %ld KB\n",
               memory_kb);
    }

    printf("\nAll thread tests completed successfully!\n");

    return 0;
}
```

2. How to run in VS Code (Ubuntu WSL)

Open the Ubuntu terminal in VS Code and execute these commands:

```
# Create practical file
nano practical13.c

# Compile the program
gcc practical13.c -o practical13 -pthread

# Run the program
./practical13
```

Save the file in Nano:

- Press `Ctrl + O`, then Enter.
- Press `Ctrl + X` to exit

14. Develop programs using semaphores and mutex locks to solve synchronization problems such as Producer–Consumer. Evaluate the effect of synchronization on resource allocation and system performance.

Save the file as `practical14.c`.

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define SIZE 5
#define ITEMS 10

int buffer[SIZE];
```

```
int in = 0, out = 0;

sem_t empty, full;
pthread_mutex_t mutex;

void *producer(void *arg)
{
    for (int i = 1; i <= ITEMS; i++)
    {
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        buffer[in] = i;

        printf("Producer produced: %d\n", i);

        in = (in + 1) % SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&full);

        sleep(1);
    }

    return NULL;
}

void *consumer(void *arg)
{
    for (int i = 1; i <= ITEMS; i++)
    {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        int item = buffer[out];

        printf("Consumer consumed: %d\n", item);

        out = (out + 1) % SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);

        sleep(2);
    }

    return NULL;
}

int main()
{
    pthread_t p, c;

    // Initialize semaphores
    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
```

```
    // Initialize mutex
    pthread_mutex_init(&mutex, NULL);

    printf("PRODUCER-CONSUMER PROBLEM\n\n");

    // Create producer and consumer threads
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    // Wait for both threads
    pthread_join(p, NULL);
    pthread_join(c, NULL);

    // Destroy synchronization objects
    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    printf("\nProducer-Consumer completed successfully!\n");

    return 0;
}
```

2. How to run in VS Code (Ubuntu WSL)

Execute these commands in the Ubuntu terminal:

```
nano practical14.c

gcc practical14.c -o practical14 -pthread

./practical14
```

15. Develop programs using semaphores and mutex locks to solve synchronization problems such as Readers-Writers. Evaluate the effect of synchronization on resource allocation and system performance.

Save the file as `practical15.c`.

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

int read_count = 0;
int shared_data = 100;

sem_t write_lock;
```

```
pthread_mutex_t mutex;

void *reader(void *arg)
{
    int id = *(int *)arg;

    // Update reader count safely
    pthread_mutex_lock(&mutex);

    read_count++;

    if (read_count == 1)
    {
        sem_wait(&write_lock);
    }

    pthread_mutex_unlock(&mutex);

    // Reading shared data
    printf("Reader %d is reading data: %d\n",
           id, shared_data);

    sleep(1);

    // Decrease reader count safely
    pthread_mutex_lock(&mutex);

    read_count--;

    if (read_count == 0)
    {
        sem_post(&write_lock);
    }

    pthread_mutex_unlock(&mutex);

    printf("Reader %d completed reading.\n", id);

    return NULL;
}

void *writer(void *arg)
{
    int id = *(int *)arg;

    sem_wait(&write_lock);

    // Writing shared data
    shared_data += 10;

    printf("Writer %d is writing data: %d\n",
           id, shared_data);

    sleep(2);

    printf("Writer %d completed writing.\n", id);
```

```
    sem_post(&write_lock);

    return NULL;
}

int main()
{
    pthread_t readers[3], writers[2];

    int reader_id[3] = {1, 2, 3};
    int writer_id[2] = {1, 2};

    // Initialize synchronization variables
    sem_init(&write_lock, 0, 1);
    pthread_mutex_init(&mutex, NULL);

    printf("READERS-WRITERS PROBLEM\n\n");

    // Create reader threads
    for (int i = 0; i < 3; i++)
    {
        pthread_create(&readers[i], NULL,
                       reader, &reader_id[i]);
    }

    // Create writer threads
    for (int i = 0; i < 2; i++)
    {
        pthread_create(&writers[i], NULL,
                       writer, &writer_id[i]);
    }

    // Wait for reader threads
    for (int i = 0; i < 3; i++)
    {
        pthread_join(readers[i], NULL);
    }

    // Wait for writer threads
    for (int i = 0; i < 2; i++)
    {
        pthread_join(writers[i], NULL);
    }

    printf("\nFinal Shared Data: %d\n", shared_data);
    printf("Readers-Writers completed successfully!\n");

    // Destroy synchronization variables
    sem_destroy(&write_lock);
    pthread_mutex_destroy(&mutex);

    return 0;
}
```

2. How to run in VS Code (Ubuntu WSL)

Execute these commands in the Ubuntu terminal:

```
nano practical15.c

gcc practical15.c -o practical15 -pthread

./practical15
```

To save the file in Nano:

1. Press `Ctrl + O`.
2. Press Enter.
3. Press `Ctrl + X`.

16. Implement Banker's Algorithm and deadlock detection techniques. Analyze resource allocation patterns and evaluate system performance under different deadlock scenarios.

Save the file as `practical16.c`.

```
#include <stdio.h>

#define P 5
#define R 3

// Banker's Algorithm
void bankers()
{
    int allocation[P][R] = {
        {0, 1, 0},
        {2, 0, 0},
        {3, 0, 2},
        {2, 1, 1},
        {0, 0, 2}
    };

    int maximum[P][R] = {
        {7, 5, 3},
        {3, 2, 2},
        {9, 0, 2},
        {2, 2, 2},
        {4, 3, 3}
    };

    int available[R] = {3, 3, 2};

    int need[P][R];
    int finish[P] = {0};
    int safeSequence[P];
```

```
int count = 0;

printf("\n===== BANKER'S ALGORITHM =====\n");

// Calculate Need Matrix
printf("\nNeed Matrix:\n");

for (int i = 0; i < P; i++)
{
    for (int j = 0; j < R; j++)
    {
        need[i][j] =
            maximum[i][j] - allocation[i][j];

        printf("%d ", need[i][j]);
    }
    printf("\n");
}

// Find safe sequence
int work[R];

for (int i = 0; i < R; i++)
    work[i] = available[i];

while (count < P)
{
    int found = 0;

    for (int i = 0; i < P; i++)
    {
        if (finish[i] == 0)
        {
            int j;

            for (j = 0; j < R; j++)
            {
                if (need[i][j] > work[j])
                    break;
            }

            if (j == R)
            {
                for (int k = 0; k < R; k++)
                    work[k] += allocation[i][k];

                safeSequence[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }

    if (found == 0)
        break;
}
```

```
    if (count == P)
    {
        printf("\nSystem is in SAFE state.\n");
        printf("Safe Sequence: ");

        for (int i = 0; i < P; i++)
            printf("P%d ", safeSequence[i]);

        printf("\n");
    }
    else
    {
        printf("\nSystem is in UNSAFE state.\n");
    }
}

// Deadlock Detection Algorithm
void deadlockDetection()
{
    int allocation[P][R] = {
        {0, 1, 0},
        {2, 0, 0},
        {3, 0, 2},
        {2, 1, 1},
        {0, 0, 2}
    };

    int request[P][R] = {
        {0, 0, 0},
        {2, 0, 2},
        {0, 0, 0},
        {1, 0, 0},
        {0, 0, 2}
    };

    int available[R] = {0, 0, 0};

    int work[R];
    int finish[P] = {0};

    printf("\n===== DEADLOCK DETECTION =====\n");

    for (int i = 0; i < R; i++)
        work[i] = available[i];

    // Processes holding no resources can finish
    for (int i = 0; i < P; i++)
    {
        int hasResource = 0;

        for (int j = 0; j < R; j++)
        {
            if (allocation[i][j] != 0)
                hasResource = 1;
        }
```

```
        if (hasResource == 0)
            finish[i] = 1;
    }

    // Check whether processes can complete
    int found;

    do
    {
        found = 0;

        for (int i = 0; i < P; i++)
        {
            if (finish[i] == 0)
            {
                int j;

                for (j = 0; j < R; j++)
                {
                    if (request[i][j] > work[j])
                        break;
                }

                if (j == R)
                {
                    for (int k = 0; k < R; k++)
                        work[k] += allocation[i][k];

                    finish[i] = 1;
                    found = 1;
                }
            }
        }

    } while (found);

    int deadlock = 0;

    for (int i = 0; i < P; i++)
    {
        if (finish[i] == 0)
        {
            printf("Deadlock detected: P%d\n", i);
            deadlock = 1;
        }
    }

    if (deadlock == 0)
        printf("No deadlock detected.\n");
}

int main()
{
    printf("OPERATING SYSTEM - PRACTICAL 16\n");

    bankers();
```

```
    deadlockDetection();

    return 0;
}
```

2. Run in Ubuntu WSL (without VS Code)

Open Ubuntu from the Windows Start Menu.

Step 1: Create the file

```
nano practical16.c
```

Step 2: Paste the complete C program and save it.

- Press `Ctrl + O`
- Press Enter
- Press `Ctrl + X`

Step 3: Compile

```
gcc practical16.c -o practical16
```

Step 4: Run

```
./practical16
```

If GCC is not installed:

```
sudo apt update
sudo apt install gcc -y
```

17. Simulate paging, segmentation, and virtual memory management. Implement FIFO and LRU page replacement algorithms and compare their effectiveness using performance metrics.

Save the file as `practical17.c`.

```
#include <stdio.h>

#define MAX 30
#define FRAMES 3

// FIFO Page Replacement
void FIFO(int pages[], int n)
```

```
{
    int frames[FRAMES];
    int index = 0, faults = 0, hits = 0;

    for (int i = 0; i < FRAMES; i++)
        frames[i] = -1;

    printf("\n===== FIFO PAGE REPLACEMENT =====\n");

    for (int i = 0; i < n; i++)
    {
        int found = 0;

        for (int j = 0; j < FRAMES; j++)
        {
            if (frames[j] == pages[i])
                found = 1;
        }

        if (found)
        {
            hits++;
            printf("Page %d: HIT\n", pages[i]);
        }
        else
        {
            frames[index] = pages[i];
            index = (index + 1) % FRAMES;
            faults++;

            printf("Page %d: PAGE FAULT\n", pages[i]);
        }

        printf("Frames: ");

        for (int j = 0; j < FRAMES; j++)
        {
            if (frames[j] == -1)
                printf("- ");
            else
                printf("%d ", frames[j]);
        }

        printf("\n");
    }

    printf("\nFIFO Results:\n");
    printf("Page Faults = %d\n", faults);
    printf("Page Hits = %d\n", hits);
    printf("Hit Ratio = %.2f%%\n", (hits * 100.0) / n);
    printf("Fault Ratio = %.2f%%\n", (faults * 100.0) / n);
}

// LRU Page Replacement
void LRU(int pages[], int n)
{
    int frames[FRAMES];
```

```
int lastUsed[FRAMES];
int faults = 0, hits = 0;

for (int i = 0; i < FRAMES; i++)
{
    frames[i] = -1;
    lastUsed[i] = -1;
}

printf("\n===== LRU PAGE REPLACEMENT =====\n");

for (int i = 0; i < n; i++)
{
    int found = -1;

    for (int j = 0; j < FRAMES; j++)
    {
        if (frames[j] == pages[i])
            found = j;
    }

    if (found != -1)
    {
        hits++;
        lastUsed[found] = i;

        printf("Page %d: HIT\n", pages[i]);
    }
    else
    {
        int replace = -1;

        for (int j = 0; j < FRAMES; j++)
        {
            if (frames[j] == -1)
            {
                replace = j;
                break;
            }
        }

        if (replace == -1)
        {
            replace = 0;

            for (int j = 1; j < FRAMES; j++)
            {
                if (lastUsed[j] < lastUsed[replace])
                    replace = j;
            }
        }

        frames[replace] = pages[i];
        lastUsed[replace] = i;
        faults++;

        printf("Page %d: PAGE FAULT\n", pages[i]);
```

```
        }

        printf("Frames: ");

        for (int j = 0; j < FRAMES; j++)
        {
            if (frames[j] == -1)
                printf("- ");
            else
                printf("%d ", frames[j]);
        }

        printf("\n");
    }

    printf("\nLRU Results:\n");
    printf("Page Faults = %d\n", faults);
    printf("Page Hits = %d\n", hits);
    printf("Hit Ratio = %.2f%%\n", (hits * 100.0) / n);
    printf("Fault Ratio = %.2f%%\n", (faults * 100.0) / n);
}

// Paging Demonstration
void paging()
{
    int pageSize = 100;
    int logicalAddress = 250;

    int pageNumber = logicalAddress / pageSize;
    int offset = logicalAddress % pageSize;

    printf("\n===== PAGING =====\n");
    printf("Logical Address: %d\n", logicalAddress);
    printf("Page Size: %d\n", pageSize);
    printf("Page Number: %d\n", pageNumber);
    printf("Offset: %d\n", offset);
}

// Segmentation Demonstration
void segmentation()
{
    int base[] = {1000, 2000, 3000};
    int limit[] = {500, 600, 400};

    int segment = 1;
    int offset = 200;

    printf("\n===== SEGMENTATION =====\n");

    printf("Segment Number: %d\n", segment);
    printf("Offset: %d\n", offset);

    if (offset < limit[segment])
    {
        int physicalAddress = base[segment] + offset;

        printf("Physical Address: %d\n", physicalAddress);
```

```
    }
    else
    {
        printf("Segmentation Fault!\n");
    }
}

int main()
{
    int pages[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2};
    int n = sizeof(pages) / sizeof(pages[0]);

    printf("OPERATING SYSTEM - PRACTICAL 17\n");

    paging();

    segmentation();

    FIFO(pages, n);

    LRU(pages, n);

    printf("\n===== PERFORMANCE COMPARISON =====\n");
    printf("FIFO and LRU results are shown above.\n");
    printf("Compare page faults and hit ratios.\n");

    return 0;
}
```

2. Run in Ubuntu WSL (Without VS Code)

Open your Ubuntu terminal.

Step 1: Create file

```
nano practical17.c
```

Step 2: Paste the complete program.

Save using:

- `Ctrl + O`
- Enter
- `Ctrl + X`

Step 3: Compile

```
gcc practical17.c -o practical17
```

Step 4: Execute

```
./practical17
```

18. Simulate paging, segmentation, and virtual memory management. Implement FIFO, and Optimal page replacement algorithms and compare their effectiveness using performance metrics.

Save the file as `practical18.c`.

```
#include <stdio.h>

#define FRAMES 3

// FIFO Page Replacement
void FIFO(int pages[], int n)
{
    int frames[FRAMES];
    int faults = 0, hits = 0;
    int index = 0;

    for (int i = 0; i < FRAMES; i++)
        frames[i] = -1;

    printf("\n===== FIFO PAGE REPLACEMENT =====\n");

    for (int i = 0; i < n; i++)
    {
        int found = 0;

        for (int j = 0; j < FRAMES; j++)
        {
            if (frames[j] == pages[i])
                found = 1;
        }

        if (found)
        {
            hits++;
            printf("Page %d: HIT\n", pages[i]);
        }
        else
        {
            frames[index] = pages[i];
            index = (index + 1) % FRAMES;
            faults++;

            printf("Page %d: PAGE FAULT\n", pages[i]);
        }

        printf("Frames: ");

        for (int j = 0; j < FRAMES; j++)
        {
            if (frames[j] == -1)
```

```
                printf("- ");
            else
                printf("%d ", frames[j]);
        }

        printf("\n");
    }

    printf("\nFIFO Results:\n");
    printf("Page Faults = %d\n", faults);
    printf("Page Hits = %d\n", hits);
    printf("Hit Ratio = %.2f%%\n", hits * 100.0 / n);
    printf("Fault Ratio = %.2f%%\n", faults * 100.0 / n);
}

// Optimal Page Replacement
void Optimal(int pages[], int n)
{
    int frames[FRAMES];
    int faults = 0, hits = 0;

    for (int i = 0; i < FRAMES; i++)
        frames[i] = -1;

    printf("\n===== OPTIMAL PAGE REPLACEMENT =====\n");

    for (int i = 0; i < n; i++)
    {
        int found = 0;

        for (int j = 0; j < FRAMES; j++)
        {
            if (frames[j] == pages[i])
                found = 1;
        }

        if (found)
        {
            hits++;
            printf("Page %d: HIT\n", pages[i]);
        }
        else
        {
            int replace = -1;

            // Find an empty frame
            for (int j = 0; j < FRAMES; j++)
            {
                if (frames[j] == -1)
                {
                    replace = j;
                    break;
                }
            }

            // Find the page used farthest in the future
            if (replace == -1)
```

```
            {
                int farthest = -1;

                for (int j = 0; j < FRAMES; j++)
                {
                    int k;

                    for (k = i + 1; k < n; k++)
                    {
                        if (frames[j] == pages[k])
                            break;
                    }

                    if (k == n)
                    {
                        replace = j;
                        break;
                    }

                    if (k > farthest)
                    {
                        farthest = k;
                        replace = j;
                    }
                }
            }

            frames[replace] = pages[i];
            faults++;

            printf("Page %d: PAGE FAULT\n", pages[i]);
        }

        printf("Frames: ");

        for (int j = 0; j < FRAMES; j++)
        {
            if (frames[j] == -1)
                printf("- ");
            else
                printf("%d ", frames[j]);
        }

        printf("\n");
    }

    printf("\nOptimal Results:\n");
    printf("Page Faults = %d\n", faults);
    printf("Page Hits = %d\n", hits);
    printf("Hit Ratio = %.2f%%\n", hits * 100.0 / n);
    printf("Fault Ratio = %.2f%%\n", faults * 100.0 / n);
}

// Paging Demonstration
void paging()
{
    int logicalAddress = 250;
```

```
    int pageSize = 100;

    int pageNumber = logicalAddress / pageSize;
    int offset = logicalAddress % pageSize;

    printf("\n===== PAGING =====\n");
    printf("Logical Address: %d\n", logicalAddress);
    printf("Page Number: %d\n", pageNumber);
    printf("Offset: %d\n", offset);
}

// Segmentation Demonstration
void segmentation()
{
    int base[] = {1000, 2000, 3000};
    int limit[] = {500, 600, 400};

    int segment = 1;
    int offset = 200;

    printf("\n===== SEGMENTATION =====\n");
    printf("Segment Number: %d\n", segment);
    printf("Offset: %d\n", offset);

    if (offset < limit[segment])
    {
        printf("Physical Address: %d\n",
               base[segment] + offset);
    }
    else
    {
        printf("Segmentation Fault!\n");
    }
}

int main()
{
    int pages[] = {7, 0, 1, 2, 0, 3, 0,
                   4, 2, 3, 0, 3, 2};

    int n = sizeof(pages) / sizeof(pages[0]);

    printf("OPERATING SYSTEM - PRACTICAL 18\n");

    paging();

    segmentation();

    FIFO(pages, n);

    Optimal(pages, n);

    printf("\n===== PERFORMANCE COMPARISON =====\n");
    printf("Compare FIFO and Optimal results above.\n");

    return 0;
}
```

2. Run in Ubuntu WSL (Without VS Code)

Open Ubuntu from the Windows Start Menu.

Step 1: Create file

```
nano practical18.c
```

Step 2: Paste the complete C program.

Save using:

- `Ctrl + O`
- Press Enter.
- `Ctrl + X`

Step 3: Compile

```
gcc practical18.c -o practical18
```

Step 4: Run

```
./practical18
```

19. Simulate paging, segmentation, and virtual memory management. Implement LRU and Optimal page replacement algorithms and compare their effectiveness using performance metrics.

Save the file as `practical19.c`.

```
#include <stdio.h>

#define FRAMES 3

// LRU Page Replacement
void LRU(int pages[], int n)
{
    int frames[FRAMES];
    int lastUsed[FRAMES];
    int faults = 0, hits = 0;

    for (int i = 0; i < FRAMES; i++)
    {
        frames[i] = -1;
        lastUsed[i] = -1;
    }

    printf("\n===== LRU PAGE REPLACEMENT =====\n");
```

```
for (int i = 0; i < n; i++)
{
    int found = -1;

    for (int j = 0; j < FRAMES; j++)
    {
        if (frames[j] == pages[i])
            found = j;
    }

    if (found != -1)
    {
        hits++;
        lastUsed[found] = i;

        printf("Page %d: HIT\n", pages[i]);
    }
    else
    {
        int replace = -1;

        // Find an empty frame
        for (int j = 0; j < FRAMES; j++)
        {
            if (frames[j] == -1)
            {
                replace = j;
                break;
            }
        }

        // Find least recently used page
        if (replace == -1)
        {
            replace = 0;

            for (int j = 1; j < FRAMES; j++)
            {
                if (lastUsed[j] < lastUsed[replace])
                    replace = j;
            }
        }

        frames[replace] = pages[i];
        lastUsed[replace] = i;
        faults++;

        printf("Page %d: PAGE FAULT\n", pages[i]);
    }

    printf("Frames: ");

    for (int j = 0; j < FRAMES; j++)
    {
        if (frames[j] == -1)
            printf("- ");
```

```
            else
                printf("%d ", frames[j]);
        }

        printf("\n");
    }

    printf("\nLRU Results:\n");
    printf("Page Faults = %d\n", faults);
    printf("Page Hits = %d\n", hits);
    printf("Hit Ratio = %.2f%%\n", hits * 100.0 / n);
    printf("Fault Ratio = %.2f%%\n", faults * 100.0 / n);
}

// Optimal Page Replacement
void Optimal(int pages[], int n)
{
    int frames[FRAMES];
    int faults = 0, hits = 0;

    for (int i = 0; i < FRAMES; i++)
        frames[i] = -1;

    printf("\n===== OPTIMAL PAGE REPLACEMENT =====\n");

    for (int i = 0; i < n; i++)
    {
        int found = 0;

        for (int j = 0; j < FRAMES; j++)
        {
            if (frames[j] == pages[i])
                found = 1;
        }

        if (found)
        {
            hits++;
            printf("Page %d: HIT\n", pages[i]);
        }
        else
        {
            int replace = -1;

            // Find an empty frame
            for (int j = 0; j < FRAMES; j++)
            {
                if (frames[j] == -1)
                {
                    replace = j;
                    break;
                }
            }

            // Find page used farthest in future
            if (replace == -1)
            {
```

```
                int farthest = -1;

                for (int j = 0; j < FRAMES; j++)
                {
                    int k;

                    for (k = i + 1; k < n; k++)
                    {
                        if (frames[j] == pages[k])
                            break;
                    }

                    if (k == n)
                    {
                        replace = j;
                        break;
                    }

                    if (k > farthest)
                    {
                        farthest = k;
                        replace = j;
                    }
                }
            }

            frames[replace] = pages[i];
            faults++;

            printf("Page %d: PAGE FAULT\n", pages[i]);
        }

        printf("Frames: ");

        for (int j = 0; j < FRAMES; j++)
        {
            if (frames[j] == -1)
                printf("- ");
            else
                printf("%d ", frames[j]);
        }

        printf("\n");
    }

    printf("\nOptimal Results:\n");
    printf("Page Faults = %d\n", faults);
    printf("Page Hits = %d\n", hits);
    printf("Hit Ratio = %.2f%%\n", hits * 100.0 / n);
    printf("Fault Ratio = %.2f%%\n", faults * 100.0 / n);
}

// Paging Demonstration
void paging()
{
    int logicalAddress = 250;
    int pageSize = 100;
```

```
    int pageNumber = logicalAddress / pageSize;
    int offset = logicalAddress % pageSize;

    printf("\n===== PAGING =====\n");
    printf("Logical Address: %d\n", logicalAddress);
    printf("Page Number: %d\n", pageNumber);
    printf("Offset: %d\n", offset);
}

// Segmentation Demonstration
void segmentation()
{
    int base[] = {1000, 2000, 3000};
    int limit[] = {500, 600, 400};

    int segment = 1;
    int offset = 200;

    printf("\n===== SEGMENTATION =====\n");
    printf("Segment Number: %d\n", segment);
    printf("Offset: %d\n", offset);

    if (offset < limit[segment])
    {
        printf("Physical Address: %d\n",
               base[segment] + offset);
    }
    else
    {
        printf("Segmentation Fault!\n");
    }
}

int main()
{
    int pages[] = {7, 0, 1, 2, 0, 3, 0,
                   4, 2, 3, 0, 3, 2};

    int n = sizeof(pages) / sizeof(pages[0]);

    printf("OPERATING SYSTEM - PRACTICAL 19\n");

    paging();

    segmentation();

    LRU(pages, n);

    Optimal(pages, n);

    printf("\n===== PERFORMANCE COMPARISON =====\n");
    printf("Compare LRU and Optimal results above.\n");

    return 0;
}
```

2. Run in Ubuntu WSL (Without VS Code)

Open Ubuntu from the Windows Start Menu.

Step 1: Create the file

```
nano practical19.c
```

Step 2: Paste the complete C program.

Save using:

- `Ctrl + O`
- Press Enter.
- `Ctrl + X`

Step 3: Compile

```
gcc practical19.c -o practical19
```

Step 4: Run

```
./practical19
```

20. Implement file allocation and disk scheduling algorithms (FCFS, SSTF). Analyze storage utilization, access time, and system throughput.

Save the file as `practical20.c`.

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 50

// Contiguous File Allocation
void contiguous()
{
    int start, length;

    printf("\n===== CONTIGUOUS FILE ALLOCATION =====\n");

    printf("Enter starting block: ");
    scanf("%d", &start);

    printf("Enter file length (number of blocks): ");
    scanf("%d", &length);
```

```
    printf("Allocated blocks: ");

    for (int i = 0; i < length; i++)
        printf("%d ", start + i);

    printf("\n");
}

// Linked File Allocation
void linked()
{
    int n, blocks[MAX];

    printf("\n===== LINKED FILE ALLOCATION =====\n");

    printf("Enter number of blocks: ");
    scanf("%d", &n);

    printf("Enter block numbers:\n");

    for (int i = 0; i < n; i++)
        scanf("%d", &blocks[i]);

    printf("Linked allocation: ");

    for (int i = 0; i < n; i++)
    {
        printf("%d", blocks[i]);

        if (i < n - 1)
            printf(" -> ");
    }

    printf(" -> NULL\n");
}

// Indexed File Allocation
void indexed()
{
    int n, indexBlock, blocks[MAX];

    printf("\n===== INDEXED FILE ALLOCATION =====\n");

    printf("Enter index block: ");
    scanf("%d", &indexBlock);

    printf("Enter number of data blocks: ");
    scanf("%d", &n);

    printf("Enter data block numbers:\n");

    for (int i = 0; i < n; i++)
        scanf("%d", &blocks[i]);

    printf("Index Block: %d\n", indexBlock);
    printf("Data blocks: ");
```

```
    for (int i = 0; i < n; i++)
        printf("%d ", blocks[i]);

    printf("\n");
}

// FCFS Disk Scheduling
void FCFS(int request[], int n, int head)
{
    int movement = 0;
    int current = head;

    printf("\n===== FCFS DISK SCHEDULING =====\n");

    printf("Head movement: %d", head);

    for (int i = 0; i < n; i++)
    {
        movement += abs(request[i] - current);
        current = request[i];

        printf(" -> %d", request[i]);
    }

    printf("\nTotal Head Movement = %d\n", movement);
    printf("Average Seek Distance = %.2f\n",
           (float)movement / n);
}

// SSTF Disk Scheduling
void SSTF(int request[], int n, int head)
{
    int visited[MAX] = {0};
    int movement = 0;
    int current = head;

    printf("\n===== SSTF DISK SCHEDULING =====\n");

    printf("Head movement: %d", head);

    for (int i = 0; i < n; i++)
    {
        int min = 100000;
        int index = -1;

        for (int j = 0; j < n; j++)
        {
            if (!visited[j])
            {
                int distance = abs(request[j] - current);

                if (distance < min)
                {
                    min = distance;
                    index = j;
                }
            }
```

```
        }

        visited[index] = 1;
        movement += min;
        current = request[index];

        printf(" -> %d", current);
    }

    printf("\nTotal Head Movement = %d\n", movement);
    printf("Average Seek Distance = %.2f\n",
           (float)movement / n);
}

int main()
{
    int request[] = {98, 183, 37, 122, 14, 124, 65, 67};
    int n = 8;
    int head = 53;

    printf("OPERATING SYSTEM - PRACTICAL 20\n");

    // File Allocation
    contiguous();

    linked();

    indexed();

    // Disk Scheduling
    printf("\nInitial Disk Head Position: %d\n", head);

    printf("Disk Requests: ");
    for (int i = 0; i < n; i++)
        printf("%d ", request[i]);

    printf("\n");

    FCFS(request, n, head);

    SSTF(request, n, head);

    printf("\n===== PERFORMANCE COMPARISON =====\n");
    printf("FCFS and SSTF results displayed above.\n");

    return 0;
}
```

2. Run in Ubuntu WSL (Without VS Code)

Open Ubuntu from the Windows Start Menu.

Step 1: Create file

```
nano practical20.c
```

Step 2: Paste the complete program.

Save using:

- `Ctrl + O`
- Press Enter.
- `Ctrl + X`

Step 3: Compile

```
gcc practical20.c -o practical20
```

Step 4: Run

```
./practical20
```

21. Implement file allocation and disk scheduling algorithms (FCFS, SCAN). Analyze storage utilization, access time, and system throughput.

Save the file as `practical21.c`.

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 50

// Contiguous File Allocation
void contiguous()
{
    int start, length;

    printf("\n===== CONTIGUOUS FILE ALLOCATION =====\n");

    printf("Enter starting block: ");
    scanf("%d", &start);

    printf("Enter file length: ");
    scanf("%d", &length);

    printf("Allocated blocks: ");

    for (int i = 0; i < length; i++)
        printf("%d ", start + i);

    printf("\n");
}

// Linked File Allocation
```

```
void linked()
{
    int n, blocks[MAX];

    printf("\n===== LINKED FILE ALLOCATION =====\n");

    printf("Enter number of blocks: ");
    scanf("%d", &n);

    printf("Enter block numbers:\n");

    for (int i = 0; i < n; i++)
        scanf("%d", &blocks[i]);

    printf("Linked allocation: ");

    for (int i = 0; i < n; i++)
    {
        printf("%d", blocks[i]);

        if (i < n - 1)
            printf(" -> ");
    }

    printf(" -> NULL\n");
}

// Indexed File Allocation
void indexed()
{
    int n, indexBlock, blocks[MAX];

    printf("\n===== INDEXED FILE ALLOCATION =====\n");

    printf("Enter index block: ");
    scanf("%d", &indexBlock);

    printf("Enter number of data blocks: ");
    scanf("%d", &n);

    printf("Enter data block numbers:\n");

    for (int i = 0; i < n; i++)
        scanf("%d", &blocks[i]);

    printf("Index Block: %d\n", indexBlock);
    printf("Data blocks: ");

    for (int i = 0; i < n; i++)
        printf("%d ", blocks[i]);

    printf("\n");
}

// FCFS Disk Scheduling
void FCFS(int request[], int n, int head)
{
```

```
    int movement = 0;
    int current = head;

    printf("\n===== FCFS DISK SCHEDULING =====\n");

    printf("Head movement: %d", head);

    for (int i = 0; i < n; i++)
    {
        movement += abs(request[i] - current);
        current = request[i];

        printf(" -> %d", request[i]);
    }

    printf("\nTotal Head Movement = %d\n", movement);
    printf("Average Seek Distance = %.2f\n",
           (float)movement / n);
}

// SCAN Disk Scheduling
void SCAN(int request[], int n, int head, int diskSize)
{
    int sorted[MAX];
    int movement = 0;
    int current = head;

    // Copy requests
    for (int i = 0; i < n; i++)
        sorted[i] = request[i];

    // Sort requests in ascending order
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (sorted[j] > sorted[j + 1])
            {
                int temp = sorted[j];
                sorted[j] = sorted[j + 1];
                sorted[j + 1] = temp;
            }
        }
    }

    printf("\n===== SCAN DISK SCHEDULING =====\n");
    printf("Head movement: %d", head);

    // Move towards higher cylinder numbers
    int i;

    for (i = 0; i < n; i++)
    {
        if (sorted[i] >= head)
            break;
    }
```

```
    for (int j = i; j < n; j++)
    {
        movement += abs(sorted[j] - current);
        current = sorted[j];

        printf(" -> %d", current);
    }

    // Reach the end of the disk
    movement += abs((diskSize - 1) - current);
    current = diskSize - 1;

    printf(" -> %d", current);

    // Reverse direction
    for (int j = i - 1; j >= 0; j--)
    {
        movement += abs(sorted[j] - current);
        current = sorted[j];

        printf(" -> %d", current);
    }

    printf("\nTotal Head Movement = %d\n", movement);
    printf("Average Seek Distance = %.2f\n",
           (float)movement / n);
}

int main()
{
    int request[] = {98, 183, 37, 122, 14, 124, 65, 67};

    int n = 8;
    int head = 53;
    int diskSize = 200;

    printf("OPERATING SYSTEM - PRACTICAL 21\n");

    // File Allocation
    contiguous();
    linked();
    indexed();

    // Disk Scheduling
    printf("\nInitial Head Position: %d\n", head);
    printf("Disk Size: %d cylinders\n", diskSize);

    printf("Disk Requests: ");

    for (int i = 0; i < n; i++)
        printf("%d ", request[i]);

    printf("\n");

    FCFS(request, n, head);

    SCAN(request, n, head, diskSize);
```

```
    printf("\n===== PERFORMANCE COMPARISON =====\n");
    printf("FCFS and SCAN results displayed above.\n");

    return 0;
}
```

2. Run in Ubuntu WSL (Without VS Code)

Open Ubuntu from the Windows Start Menu.

Step 1: Create the file

```
nano practical21.c
```

Step 2: Paste the complete C program.

Save using:

- `Ctrl + O`
- Press Enter.
- `Ctrl + X`

Step 3: Compile

```
gcc practical21.c -o practical21
```

Step 4: Run

```
./practical21
```

22. Implement file allocation and disk scheduling algorithms (FCFS, CSCAN). Analyze storage utilization, access time, and system throughput.

Save the file as `practical22.c`.

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 50

// Contiguous File Allocation
void contiguous()
{
    int start, length;

    printf("\n===== CONTIGUOUS FILE ALLOCATION =====\n");
```

```
    printf("Enter starting block: ");
    scanf("%d", &start);

    printf("Enter file length: ");
    scanf("%d", &length);

    printf("Allocated blocks: ");

    for (int i = 0; i < length; i++)
        printf("%d ", start + i);

    printf("\n");
}

// Linked File Allocation
void linked()
{
    int n, blocks[MAX];

    printf("\n===== LINKED FILE ALLOCATION =====\n");

    printf("Enter number of blocks: ");
    scanf("%d", &n);

    printf("Enter block numbers:\n");

    for (int i = 0; i < n; i++)
        scanf("%d", &blocks[i]);

    printf("Linked allocation: ");

    for (int i = 0; i < n; i++)
    {
        printf("%d", blocks[i]);

        if (i < n - 1)
            printf(" -> ");
    }

    printf(" -> NULL\n");
}

// Indexed File Allocation
void indexed()
{
    int n, indexBlock, blocks[MAX];

    printf("\n===== INDEXED FILE ALLOCATION =====\n");

    printf("Enter index block: ");
    scanf("%d", &indexBlock);

    printf("Enter number of data blocks: ");
    scanf("%d", &n);

    printf("Enter data block numbers:\n");
```

```
    for (int i = 0; i < n; i++)
        scanf("%d", &blocks[i]);

    printf("Index Block: %d\n", indexBlock);
    printf("Data blocks: ");

    for (int i = 0; i < n; i++)
        printf("%d ", blocks[i]);

    printf("\n");
}

// FCFS Disk Scheduling
void FCFS(int request[], int n, int head)
{
    int movement = 0;
    int current = head;

    printf("\n===== FCFS DISK SCHEDULING =====\n");

    printf("Head movement: %d", head);

    for (int i = 0; i < n; i++)
    {
        movement += abs(request[i] - current);
        current = request[i];

        printf(" -> %d", request[i]);
    }

    printf("\nTotal Head Movement = %d\n", movement);
    printf("Average Seek Distance = %.2f\n",
           (float)movement / n);
}

// C-SCAN Disk Scheduling
void CSCAN(int request[], int n, int head, int diskSize)
{
    int sorted[MAX];
    int movement = 0;
    int current = head;

    // Copy requests
    for (int i = 0; i < n; i++)
        sorted[i] = request[i];

    // Sort requests in ascending order
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (sorted[j] > sorted[j + 1])
            {
                int temp = sorted[j];
                sorted[j] = sorted[j + 1];
                sorted[j + 1] = temp;
            }
```

```
        }
    }

    printf("\n===== C-SCAN DISK SCHEDULING =====\n");

    printf("Head movement: %d", head);

    int i;

    // Move towards higher cylinder numbers
    for (i = 0; i < n; i++)
    {
        if (sorted[i] >= head)
            break;
    }

    // Service higher requests
    for (int j = i; j < n; j++)
    {
        movement += abs(sorted[j] - current);
        current = sorted[j];

        printf(" -> %d", current);
    }

    // Move to last cylinder
    movement += abs((diskSize - 1) - current);
    current = diskSize - 1;

    printf(" -> %d", current);

    // Circular jump to first cylinder
    movement += current;
    current = 0;

    printf(" -> %d", current);

    // Service remaining lower requests
    for (int j = 0; j < i; j++)
    {
        movement += abs(sorted[j] - current);
        current = sorted[j];

        printf(" -> %d", current);
    }

    printf("\nTotal Head Movement = %d\n", movement);
    printf("Average Seek Distance = %.2f\n",
           (float)movement / n);
}

int main()
{
    int request[] = {98, 183, 37, 122, 14, 124, 65, 67};

    int n = 8;
    int head = 53;
```

```
    int diskSize = 200;

    printf("OPERATING SYSTEM - PRACTICAL 22\n");

    // File Allocation
    contiguous();
    linked();
    indexed();

    // Disk Scheduling
    printf("\nInitial Head Position: %d\n", head);
    printf("Disk Size: %d cylinders\n", diskSize);

    printf("Disk Requests: ");

    for (int i = 0; i < n; i++)
        printf("%d ", request[i]);

    printf("\n");

    FCFS(request, n, head);

    CSCAN(request, n, head, diskSize);

    printf("\n===== PERFORMANCE COMPARISON =====\n");
    printf("FCFS and C-SCAN results displayed above.\n");

    return 0;
}
```

2. Run in Ubuntu WSL (Without VS Code)

Open Ubuntu from the Windows Start Menu.

Step 1: Create the file

```
nano practical22.c
```

Step 2: Paste the complete C program.

Save using:

- `Ctrl + O`
- Press Enter.
- `Ctrl + X`

Step 3: Compile

```
gcc practical22.c -o practical22
```

Step 4: Run

./practical22

23. Deploy a simple AI/ML application inside a Docker container. Monitor CPU, memory, storage, and process utilization using Linux tools and evaluate the benefits of container-based deployment.